

Data Structures: Stacks

Minseok Jeon
DGIST

November 2, 2025

Contents

1. Introduction to Stacks
2. Core Operations
3. Implementation Approaches
4. Applications
5. Complexity Analysis
6. Summary

Introduction to Stacks

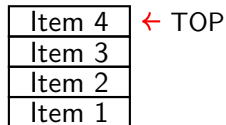
What is a Stack?

Definition

A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle.

Key characteristics:

- Elements are added and removed from the same end (top)
- Only the top element is accessible
- Perfect for managing nested operations and histories
- Natural structure for function calls and recursion



Stack Structure

Core Operations

Essential Stack Operations

Primary Operations

- **Push:** Add element to top
- **Pop:** Remove and return top element
- **Peek/Top:** Return top element without removing
- **Empty:** Check if stack is empty
- **Size:** Get number of elements

C Example

```
1 int s[100];
2 int top = -1;           // empty
3
4 // push
5 s[++top] = 10;
6 s[++top] = 20;
7
8 // peek (top element)
9 int topVal = s[top];    // -> 20
10
11 // pop
12 int x = s[top--];       // -> 20
13
14 // empty?
15 int empty = (top == -1);
```

Time Complexity

All operations are **O(1)** - constant time (amortized for push in dynamic arrays)

Stack Operations Visualization

Initial Stack

20
10

After Push(30)

30
20
10

← PUSH

After Pop()

20
10

← TOP

Returns: 30

Implementation Approaches

Array-based vs Linked List Implementation

Array-based Stack

Pros:

- Contiguous memory
- Great cache locality
- Simple implementation
- $O(1)$ amortized push

Cons:

- Occasional resize cost
- Capacity management

Linked List-based Stack

Pros:

- No resize cost
- Always $O(1)$ operations
- Dynamic size

Cons:

- Extra pointer memory
- Poor cache locality
- More complex

Linked List Stack Implementation

```
1 class Node:
2     def __init__(self, val, next=None):
3         self.val = val
4         self.next = next
5
6 class StackLL:
7     def __init__(self):
8         self.head = None
9         self.n = 0
10
11     def push(self, x):
12         self.head = Node(x, self.head)
13         self.n += 1
14
15     def pop(self):
16         if not self.head:
17             raise IndexError("pop from empty stack")
```

Applications

Expression Evaluation: Parentheses Matching

Problem

Check if parentheses, brackets, and braces are properly balanced.

Algorithm:

1. Push opening brackets onto stack
2. For closing brackets:
 - Check if stack is empty
 - Check if top matches type
 - Pop if match, return false if not
3. Stack should be empty at end

```
1 def valid_brackets(s):
2     pairs = {'(':')', '[':']', '{':'}'
3     st = []
4     for ch in s:
5         if ch in '([{':
6             st.append(ch)
7         elif ch in ')]}':
8             if not st or st[-1] != pairs[ch]:
9                 return False
10            st.pop()
11    return not st
12
13 # Examples:
14 # valid_brackets("([)]{}") -> True
15 # valid_brackets("([)]") -> False
```

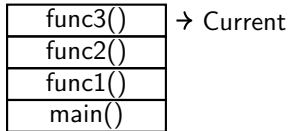
Function Call Stack and Recursion

Call Stack Mechanism

- Each function call creates a **stack frame**
- Frame contains: parameters, local variables, return address
- Recursion uses call stack implicitly
- Deep recursion can cause stack overflow

Converting Recursion to Iteration

Use explicit stack to avoid stack overflow for deep recursion



Call Stack

Undo/Redo Operations

Two-Stack Approach

- **Undo Stack:** stores performed actions
- **Redo Stack:** stores undone actions

Operations:

- **Action:** push to undo, clear redo
- **Undo:** pop from undo, apply inverse, push to redo
- **Redo:** pop from redo, apply, push to undo

Simple undo/redo pattern in pseudocode:

- `do(action)` - execute and save
- `undo()` - reverse last action
- `redo()` - replay undone action

Real-world Examples

Text editors, image editing software, IDEs, web browsers

Complexity Analysis

Time and Space Complexity

Implementation	Push	Pop/Peek	Space
Array-based	$O(1)$ amortized	$O(1)$	$O(n)$
Linked List-based	$O(1)$	$O(1)$	$O(n)$ + pointer overhead

Array-based Considerations

- Amortized $O(1)$ push due to resize
- Worst-case single push: $O(n)$
- Better cache performance

Linked List Considerations

- Guaranteed $O(1)$ for all operations
- Extra memory per node
- Dynamic allocation overhead

Summary

Key Takeaways

Stack Fundamentals

- LIFO data structure with top-only access
- Essential operations: push, pop, peek, empty, size
- All operations are $O(1)$ time complexity

Implementation Choices

- Array-based: better cache, amortized $O(1)$
- Linked list-based: guaranteed $O(1)$, more memory

Important Applications

- Expression evaluation and parentheses matching
- Function call management and recursion
- Undo/redo functionality
- Backtracking algorithms

Thank You!

Questions?